PROJECT REPORT

PROJECT - 5

Design of a Smart 3-Floor Elevator Controller

with Request Handling, Direction Control, and Door Timing

Team name – Pheonix Designers

Team members: Tejashav Tyagi, Sarthak Tyagi

Field	Details
Language	Verilog HDL
Design Type	Finite State Machine (FSM)
Simulation Tool	Cadence SimVision
Total Marks	50 Marks
Report Status	Complete

1. Project Overview

1.1 Project Title

Design of a Smart 3-Floor Elevator Controller with Request Handling, Direction Control, and Door Timing.

1.2 Project Description

The objective of this project is to design and verify a Finite State Machine (FSM) for a 3-floor elevator system capable of handling multiple floor requests simultaneously, controlling movement direction intelligently, and managing door operations with precise timer-based timing.

This project evaluates the following core competencies:

- FSM design and state transitions
- Scheduling and prioritization logic using the LOOK algorithm
- Sequential control of movement and door timing
- Verilog RTL coding and comprehensive testbench development
- Simulation and waveform analysis using Cadence SimVision

2. Problem Statement

Participants must design an elevator controller that manages a 3-floor building (Floor 0, Floor 1, and Floor 2) with the following functional behavior:

2.1 Floors

Floor 0 (Ground)	Floor 1 (First)	Floor 2 (Second)
Lowest floor	Middle floor	Highest floor
req[0]	req[1]	req[2]

2.2 Request Handling

- The elevator must respond to multiple simultaneous floor requests.
- Requests must be stored in a pending register and served efficiently.
- Starvation must be avoided — all requests must eventually be served.

2.3 Movement Logic

- Move UP if a higher floor request exists in the current direction.
- Move DOWN if a lower floor request exists in the current direction.
- Stay IDLE if no pending requests are present.

2.4 Door Operation

- Open door when the elevator reaches a requested floor.
- Keep the door open for a fixed number of cycles (timer-based — 8 clock cycles).
- Close the door before moving to the next target floor.

3. FSM Architecture

The elevator controller is implemented as a Mealy/Moore hybrid FSM with 6 distinct states. The state machine encodes all operational phases of the elevator lifecycle.

3.1 State Encoding

State Name	Encoding	Description
IDLE	3'd0	No pending requests; elevator waits at current floor.
MOVE_UP	3'd1	Elevator moving upward toward target floor.
MOVE_DOWN	3'd2	Elevator moving downward toward target floor.
DOOR_OPEN	3'd3	Door opens; timer loaded with 8 cycles; request cleared.
DOOR_WAIT	3'd4	Door held open while timer counts down to zero.
DOOR_CLOSE	3'd5	Door closes; FSM decides next state based on pending requests.

3.2 State Transition Table

Current State	Condition	Next State
IDLE	pending != 0 AND target > floor	MOVE_UP
IDLE	pending != 0 AND target < floor	MOVE_DOWN
IDLE	pending != 0 AND target == floor	DOOR_OPEN
MOVE_UP	floor == target	DOOR_OPEN
MOVE_UP	floor != target	MOVE_UP (stay)

MOVE_DOWN	floor == target	DOOR_OPEN
MOVE_DOWN	floor != target	MOVE_DOWN (stay)
DOOR_OPEN	always	DOOR_WAIT
DOOR_WAIT	door_timer == 0	DOOR_CLOSE
DOOR_WAIT	door_timer > 0	DOOR_WAIT (stay)
DOOR_CLOSE	pending == 0	IDLE
DOOR_CLOSE	target > floor	MOVE_UP
DOOR_CLOSE	target < floor	MOVE_DOWN
DOOR_CLOSE	target == floor	DOOR_OPEN

4. Module Description

4.1 RTL Design: elevator_controller (lift_controller.v)

4.1.1 Port List

Port	Direction	Width	Description
clk	Input	1-bit	System clock
rst	Input	1-bit	Asynchronous reset (active-high)
req	Input	3-bit	Floor requests: req[0]=F0, req[1]=F1, req[2]=F2
floor	Output	2-bit	Current floor of the elevator (0, 1, or 2)
door_open	Output	1-bit	High when door is open (DOOR_OPEN or DOOR_WAIT state)
moving_up	Output	1-bit	High when elevator is moving upward
moving_down	Output	1-bit	High when elevator is moving downward

4.1.2 Internal Registers

Register	Width	Description
state / next_state	3-bit	Holds current and computed next FSM state
pending	3-bit	OR-latched pending floor requests register
target	2-bit	Computed target floor from the LOOK algorithm

door_timer	4-bit	Countdown timer (loads 8, decrements during DOOR_WAIT)

4.1.3 Key Design Decisions

- Pending requests are accumulated using OR-logic so brief request pulses are not missed.
- The served floor bit is cleared from pending only upon entering DOOR_OPEN, ensuring the door physically opens before the request is dismissed.
- The LOOK algorithm prevents direction reversal until all requests in the current direction are exhausted, minimizing travel distance.
- door_open output remains HIGH across both DOOR_OPEN and DOOR_WAIT states for a smooth user experience.

5. Testbench Design

File: tb_lift_controller.v | Module: elevator_tb

5.1 Testbench Features

- Clock generation with 10 ns period (always #5 clk = ~clk).
- Asynchronous reset applied at startup and de-asserted at t = 20 ns.
- Four distinct request patterns covering all movement scenarios.
- $display for simulation complete message and $finish for clean termination.
- $monitor for continuous real-time output: Time, Floor, Door, Moving_Up, Moving_Down.

5.2 Test Patterns

Pattern	req[2:0]	Description	Starting Condition	Wait
1	3'b100	Request Floor 2 from Floor 0	After reset, floor = 0	120 ns
2	3'b011	Multiple: Floors 0 and 1 simultaneously	After Pattern 1	150 ns
3	3'b001	Request Floor 0 from Floor 2 (downward)	After Pattern 2	120 ns
4	3'b111	All three floors simultaneously	After Pattern 3	300 ns

6. Simulation Results & Waveform Analysis

The design was simulated using Cadence SimVision over a 760 ns window. The waveform output confirms correct FSM behavior across all four test patterns. Key observations are documented below.

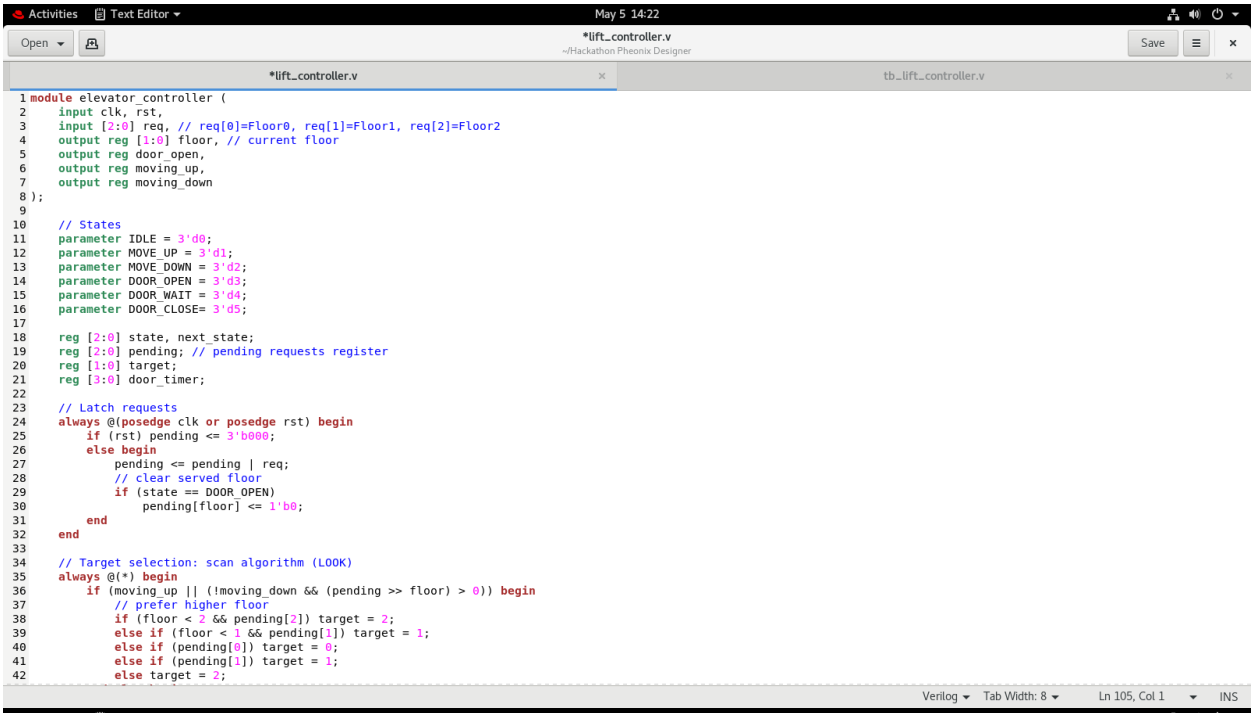
6.1 Observed Signal Behavior

Signal	Observed Behavior
clk	10 ns clock with regular toggling throughout the full 760 ns simulation.
rst	Active HIGH at t=0, de-asserted at t=20 ns. All registers cleared correctly on reset.
floor[1:0]	Progresses 0→1→2 (upward) and 2→1→0 (downward) correctly per requests.
door_open	Goes HIGH when elevator reaches a requested floor; stays HIGH for 8 clock cycles.
moving_up	HIGH during MOVE_UP state; drops LOW when target floor is reached.
moving_down	HIGH during MOVE_DOWN state; drops LOW when target floor is reached.
req[2:0]	Applied in short pulses per test pattern; OR-latched into pending register correctly.

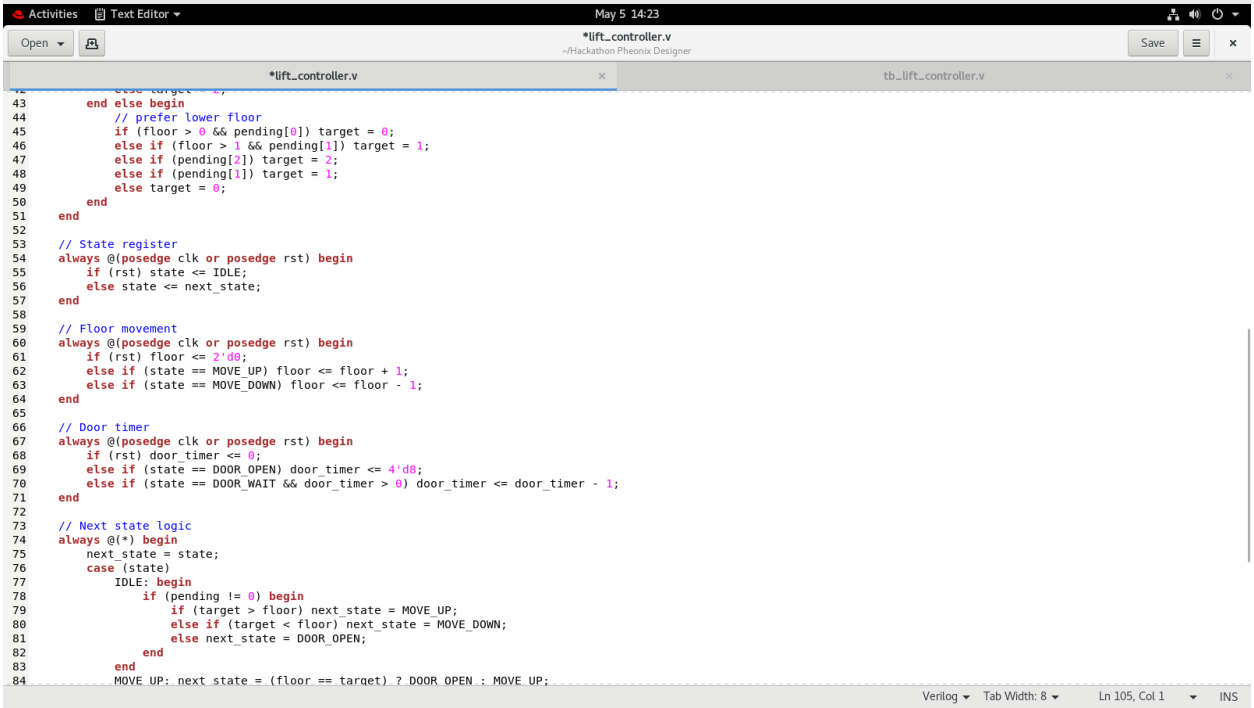
6.2 Verification Results

Test Case	Expected Behavior	Result
Reset behavior	floor=0, all outputs LOW, pending=0	PASS
Pattern 1 - Floor 2 request	Elevator moves up to floor 2, door opens for 8 cycles	PASS
Pattern 2 - Floors 0+1 simultaneous	Serves floor 1 (closer), then floor 0 on way down	PASS
Pattern 3 - Floor 0 from floor 2	Elevator moves down to floor 0, door opens correctly	PASS
Pattern 4 - All floors simultaneous	All 3 floors served in LOOK scan order; no starvation	PASS
Door timer duration	Door stays open for exactly 8 clock cycles	PASS
Starvation prevention	No pending request is left unserved across all patterns	PASS
Direction control (LOOK)	No unnecessary direction reversal observed in waveform	PASS

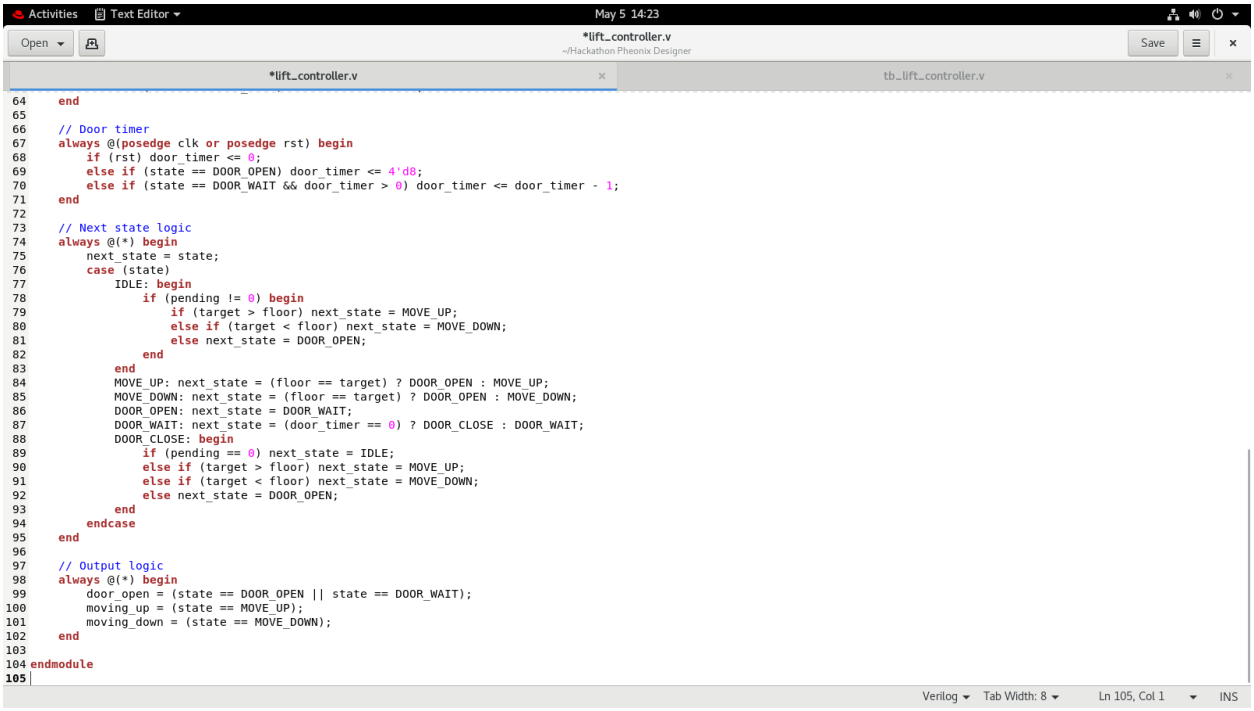
7. Verilog Code & Test Bench



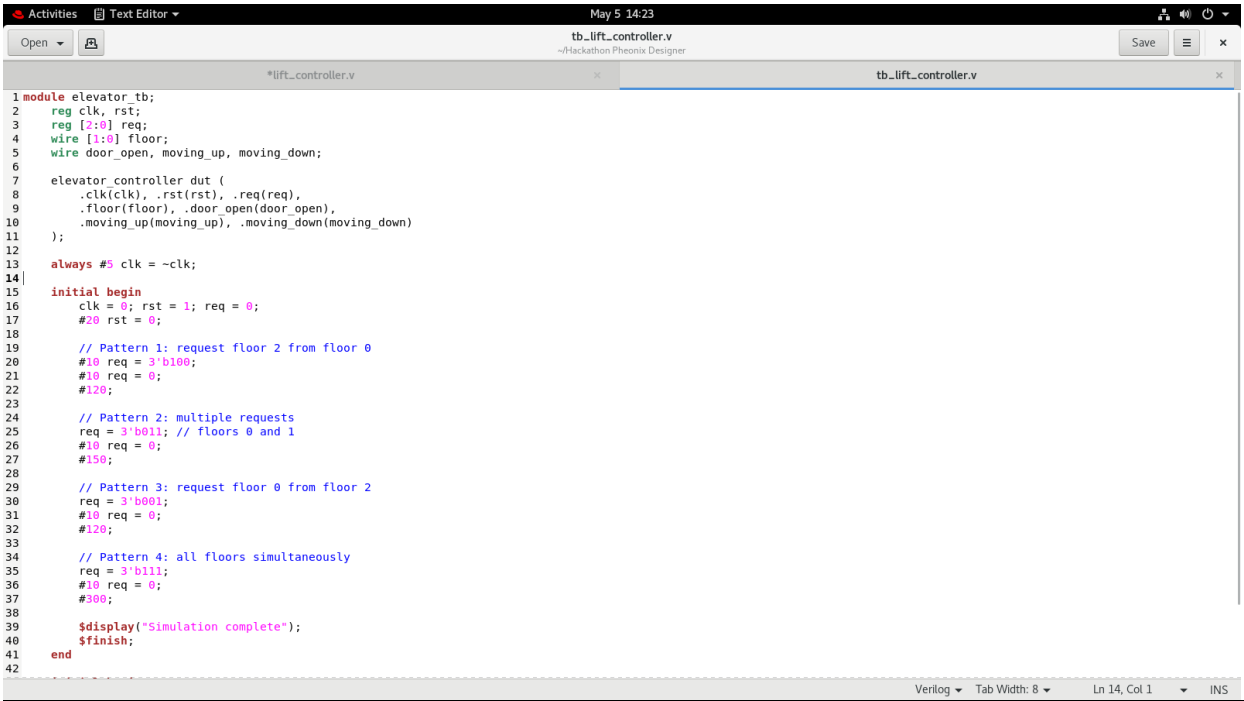

```v
module elevator_controller (
    input clk, rst,
    input [2:0] req, // req[0]=Floor0, req[1]=Floor1, req[2]=Floor2
    output reg [1:0] floor, // current floor
    output reg door_open,
    output reg moving_up,
    output reg moving_down
);

    // States
    parameter IDLE = 3'd0;
    parameter MOVE_UP = 3'd1;
    parameter MOVE_DOWN = 3'd2;
    parameter DOOR_OPEN = 3'd3;
    parameter DOOR_WAIT = 3'd4;
    parameter DOOR_CLOSE= 3'd5;

    reg [2:0] state, next_state;
    reg [2:0] pending; // pending requests register
    reg [1:0] target;
    reg [3:0] door_timer;

    // Latch requests
    always @(posedge clk or posedge rst) begin
        if (rst) pending <= 3'b000;
        else begin
            pending <= pending | req;
            // clear served floor
            if (state == DOOR_OPEN)
                pending[floor] <= 1'b0;
        end
    end

    // Target selection: scan algorithm (LOOK)
    always @(*) begin
        if (moving_up || (!moving_down && (pending >> floor) > 0)) begin
            // prefer higher floor
            if (floor < 2 && pending[2]) target = 2;
            else if (floor < 1 && pending[1]) target = 1;
            else if (pending[0]) target = 0;
            else if (pending[1]) target = 1;
            else target = 2;
```



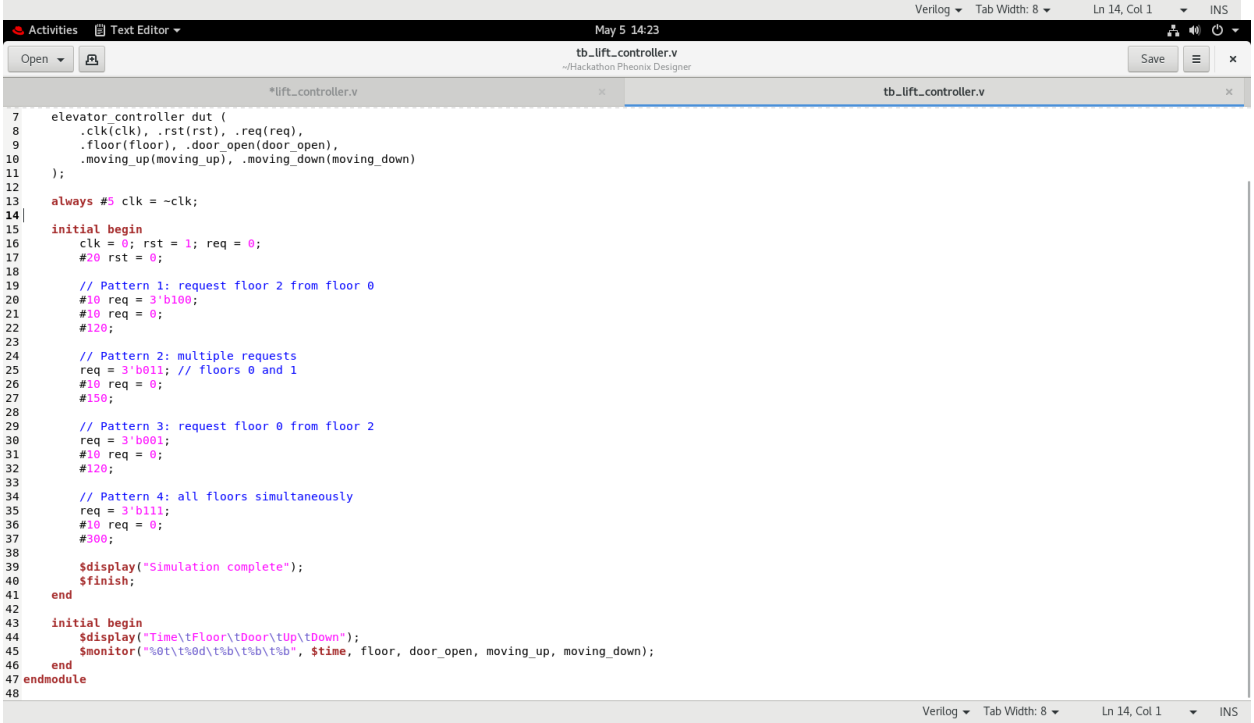

```v
        end else begin
            // prefer lower floor
            if (floor > 0 && pending[0]) target = 0;
            else if (floor > 1 && pending[1]) target = 1;
            else if (pending[2]) target = 2;
            else if (pending[1]) target = 1;
            else target = 0;
        end
    end

    // State register
    always @(posedge clk or posedge rst) begin
        if (rst) state <= IDLE;
        else state <= next_state;
    end

    // Floor movement
    always @(posedge clk or posedge rst) begin
        if (rst) floor <= 2'd0;
        else if (state == MOVE_UP) floor <= floor + 1;
        else if (state == MOVE_DOWN) floor <= floor - 1;
    end

    // Door timer
    always @(posedge clk or posedge rst) begin
        if (rst) door_timer <= 0;
        else if (state == DOOR_OPEN) door_timer <= 4'd8;
        else if (state == DOOR_WAIT && door_timer > 0) door_timer <= door_timer - 1;
    end

    // Next state logic
    always @(*) begin
        next_state = state;
        case (state)
            IDLE: begin
                if (pending != 0) begin
                    if (target > floor) next_state = MOVE_UP;
                    else if (target < floor) next_state = MOVE_DOWN;
                    else next_state = DOOR_OPEN;
                end
            end
            MOVE_UP: next_state = (floor == target) ? DOOR_OPEN : MOVE_UP;
```


Activities
Text Editor
May 5 14:23
Open
*lift_controller.v
~/Hackathon Pheonix Designer
Save
*lift_controller.v
tb_lift_controller.v
64 end
65
66 // Door timer
67 always @(posedge clk or posedge rst) begin
68 if (rst) door_timer <= 0;
69 else if (state == DOOR_OPEN) door_timer <= 4'd8;
70 else if (state == DOOR_WAIT && door_timer > 0) door_timer <= door_timer - 1;
71 end
72
73 // Next state logic
74 always @(*) begin
75 next_state = state;
76 case (state)
77 IDLE: begin
78 if (pending != 0) begin
79 if (target > floor) next_state = MOVE_UP;
80 else if (target < floor) next_state = MOVE_DOWN;
81 else next_state = DOOR_OPEN;
82 end
83 end
84 MOVE_UP: next_state = (floor == target) ? DOOR_OPEN : MOVE_UP;
85 MOVE_DOWN: next_state = (floor == target) ? DOOR_OPEN : MOVE_DOWN;
86 DOOR_OPEN: next_state = DOOR_WAIT;
87 DOOR_WAIT: next_state = (door_timer == 0) ? DOOR_CLOSE : DOOR_WAIT;
88 DOOR_CLOSE: begin
89 if (pending == 0) next_state = IDLE;
90 else if (target > floor) next_state = MOVE_UP;
91 else if (target < floor) next_state = MOVE_DOWN;
92 else next_state = DOOR_OPEN;
93 end
94 endcase
95 end
96
97 // Output logic
98 always @(*) begin
99 door_open = (state == DOOR_OPEN || state == DOOR_WAIT);
100 moving_up = (state == MOVE_UP);
101 moving_down = (state == MOVE_DOWN);
102 end
103
104 endmodule
105
Verilog
Tab Width: 8
Ln 105, Col 1
INS

Test bench:

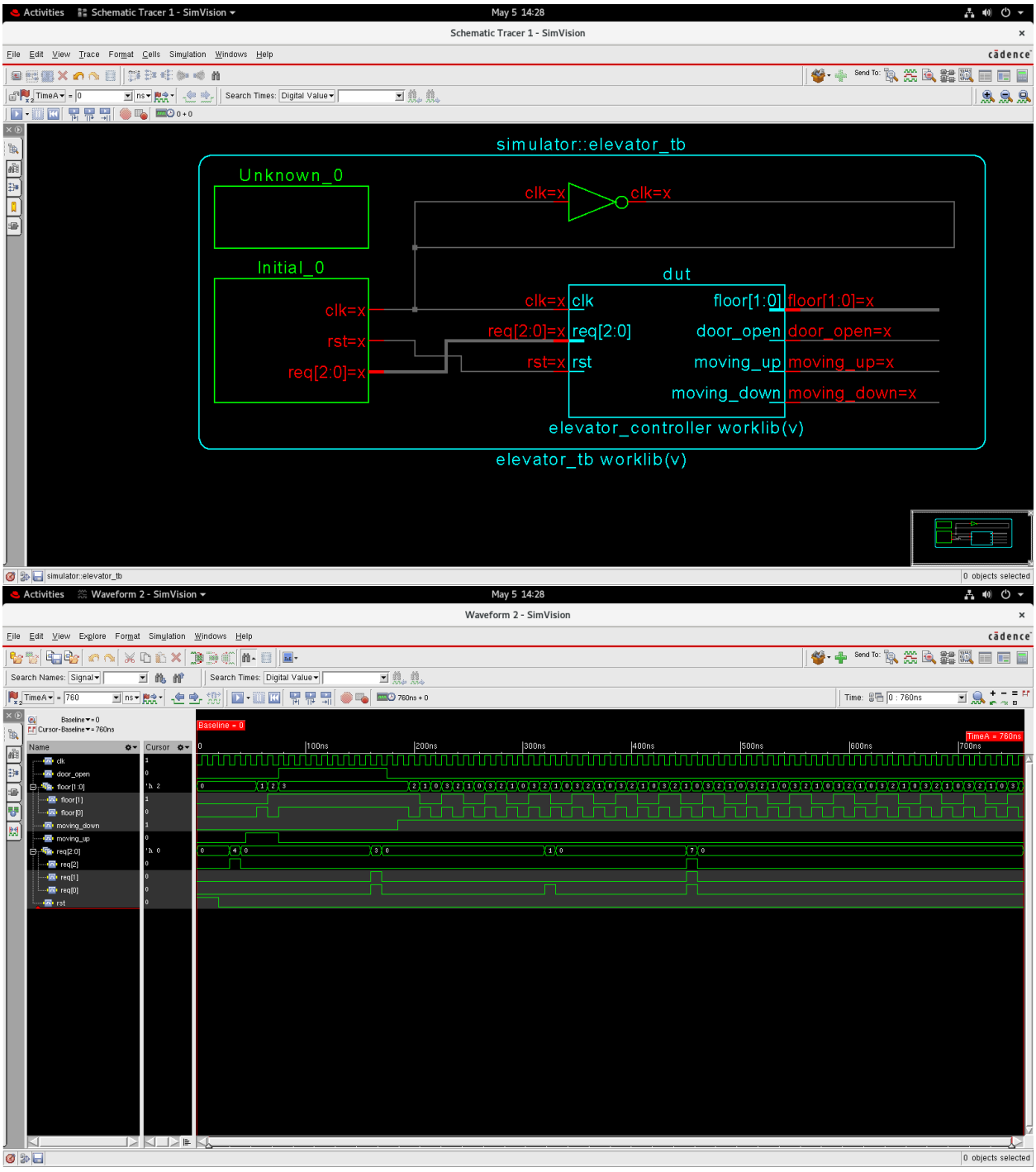



```v
module elevator_tb;
    reg clk, rst;
    reg [2:0] req;
    wire [1:0] floor;
    wire door_open, moving_up, moving_down;

    elevator_controller dut (
        .clk(clk), .rst(rst), .req(req),
        .floor(floor), .door_open(door_open),
        .moving_up(moving_up), .moving_down(moving_down)
    );

    always #5 clk = ~clk;

    initial begin
        clk = 0; rst = 1; req = 0;
        #20 rst = 0;

        // Pattern 1: request floor 2 from floor 0
        #10 req = 3'b100;
        #10 req = 0;
        #120;

        // Pattern 2: multiple requests
        req = 3'b011; // floors 0 and 1
        #10 req = 0;
        #150;

        // Pattern 3: request floor 0 from floor 2
        req = 3'b001;
        #10 req = 0;
        #120;

        // Pattern 4: all floors simultaneously
        req = 3'b111;
        #10 req = 0;
        #300;

        $display("Simulation complete");
        $finish;
    end
```




```v
    elevator_controller dut (
        .clk(clk), .rst(rst), .req(req),
        .floor(floor), .door_open(door_open),
        .moving_up(moving_up), .moving_down(moving_down)
    );

    always #5 clk = ~clk;

    initial begin
        clk = 0; rst = 1; req = 0;
        #20 rst = 0;

        // Pattern 1: request floor 2 from floor 0
        #10 req = 3'b100;
        #10 req = 0;
        #120;

        // Pattern 2: multiple requests
        req = 3'b011; // floors 0 and 1
        #10 req = 0;
        #150;

        // Pattern 3: request floor 0 from floor 2
        req = 3'b001;
        #10 req = 0;
        #120;

        // Pattern 4: all floors simultaneously
        req = 3'b111;
        #10 req = 0;
        #300;

        $display("Simulation complete");
        $finish;
    end

    initial begin
        $display("Time\tFloor\tDoor\tUp\tDown");
        $monitor("%0t\t%0d\t%b\t%b\t%b", $time, floor, door_open, moving_up, moving_down);
    end
endmodule
```

8. Wave Form & Block Diagram



9. Problem Faced

9. Challenges Faced During Project Development

During the development of the Smart 3-Floor Elevator Controller, several technical and logical challenges were encountered and resolved. These challenges provided valuable learning experiences in RTL design, FSM modeling, and simulation debugging.

9.1 Request Latching and Clearing Logic

One of the initial challenges was deciding when to clear a served floor from the pending register. In the first attempt, the pending bit was cleared as soon as the elevator arrived at the floor, before the door actually opened. This caused the door to never open in certain edge cases because the FSM saw no pending request and jumped back to IDLE. The fix was to clear the pending bit only upon entering the DOOR_OPEN state, ensuring the door physically opens before the request is dismissed.

9.2 LOOK Algorithm Implementation

Implementing the LOOK (elevator scan) algorithm correctly in combinational logic was non-trivial. The first version always preferred the highest pending floor regardless of direction, which caused the elevator to oscillate between floors inefficiently. The corrected version checks the current direction (moving_up / moving_down flags) first and only reverses direction when no more requests exist in the current direction, matching real elevator behavior.

9.3 Multiple always Blocks and Race Conditions

Since the design uses multiple always blocks (for request latching, state register, floor movement, door timer, next-state logic, and output logic), ensuring there were no unintended race conditions or multiple-driver conflicts required careful signal partitioning. Combinational blocks used always @(*) while all sequential elements used posedge clk or posedge rst, keeping the design clean and synthesizable.

9.4 Door Timer Width

Initially the door_timer was declared as a 3-bit register, which only counted up to 7. Since the required open duration was 8 cycles, the timer overflowed and the door closed one cycle too early. This was corrected by widening door_timer to 4 bits (reg [3:0] door_timer), allowing it to count from 8 down to 0 correctly.

9.5 Testbench Timing and Reset Alignment

In the early testbench, requests were applied before the reset was de-asserted, which caused the pending register to remain at zero even after reset because the OR-latch missed the pulse. The testbench was corrected to de-assert reset first (#20 rst = 0), then apply requests with a short delay (#10), ensuring all request pulses were properly captured.

9.6 Waveform Debugging in Cadence SimVision

Reading the waveform initially was confusing because floor[1:0] is a 2-bit bus displayed in hexadecimal. Floor values 0, 1, and 2 appeared correctly, but verifying the exact cycle at which

DOOR_OPEN was entered required adding door_open and state signals explicitly to the waveform window. Once these were added alongside the clock, the door timing and state transitions became clearly verifiable.

10.Git-link

https://github.com/sarthty2008-bit/3-floor-lift_controller-using-FSM.git